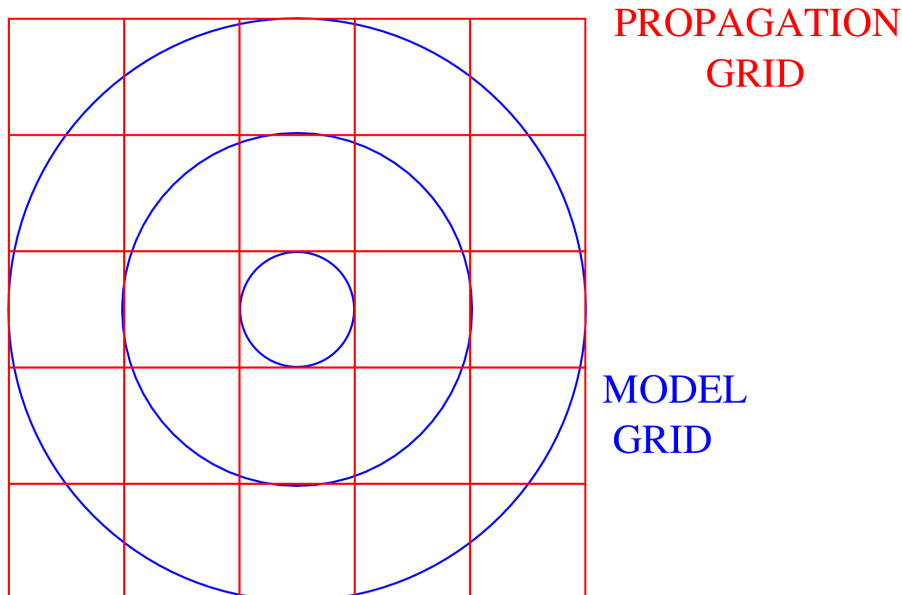


input.dat

This data are read at the beginning of the main program (main.f90).

n_pack - number of photon packages
n_nubin - number of frequency bins
nx_cell - number of propagation grid cell in x direction
ny_cell - number of propagation grid cell in y direction
nz_cell - number of propagation grid cell in z direction
model_type - type of the wind model used (1 – 1D, 2 – 2D, 3 – 3D)
xmax - coordinates of outer border of the wind in x direction in units of stellar radius
ymax - coordinates of outer border of the wind in y direction in units of stellar radius
zmax - coordinates of outer border of the wind in z direction in units of stellar radius
random seed - (or **iseed**) if **iseed** < 0 => random-seed value is random otherwise, it is fixed

modul.f90

In this module we define types and global parameters.

Type definitions:

type: grid_cell – assigned allocatable array **cell(:)** - propagation grid

cell(Ngrid)%model_index – (integer) – which model (outflow) index corresponds to every cell

cell(Ngrid)%indexc – index of each cell in x, y, and z direction ;
maximum index can be (Ngrid, Ngrid, Ngrid), I = 1, Ngrid (integer vector)
vector : cell(I)%indexc(1), cell(I)%indexc(2), cell(I)%indexc(3)

cell(Ngrid)%corner – coordinates of the lower left corner of each cell; I = 1, Ngrid (real vector)
vector : cell(I)%corner(1), cell(I)%corner(2), cell(I)%corner(3)

type: **photon** – assigned allocatable array **package(:)**

package(n_pac)%cell_numb – (integer)

package(n_pac)%pack_numb – (integer)

package(n_pac)%active – (integer)

package(n_pac)%typ – (integer)

package(n_pac)%last_cross – (integer)

package(n_pac)%pos – (real vector); **package()%pos(1)**, **package()%pos(2)**, **package()%pos(3)**

package(n_pac)%dir – (real vector); **package()%dir(1)**, **package()%dir(2)**, **package()%dir(3)**

package(n_pac)%e_cmf – (real)

package(n_pac)%e_rf – (real)

package(n_pac)%freq_cmf – (real)

package(n_pac)%freq_rf – (real)

type: **modelgrid** – assigned allocatable array **model_grid(:)** - model grid

model_grid(n_modelgrid)%rho – (real)

model_grid(n_modelgrid)%vel – (real)

model_grid(n_modelgrid)%rwind – (real)

type: **spec_type** – array with dimension **n_nubin** **spectrum(n_nubin)**

spectrum(n_nubin)%freq – (real)

spectrum(n_nubin)%flux – (real)

spectrum(n_nubin)%esc – (integer)

Different packet types:

(we give the name of the parameters and assigned values are not important, they are just put because parameters have to be defined; these parameters are called in the code by their name)

type_escaped = -99

type_rpkt = 0

type_kpkt = 1

type_ipkt = 2

rpkt_eventtype_changecell = 1

rpkt_eventtype_lineinteraction = 2

rpkt_eventtype_continuum = 3

Packet's memory of which cell surface it crossed last :

(we give the name of the parameters and assigned values are not important, they are just put because parameters have to be defined; these parameters are called in the code by their name)

posx = 1

negx = 2

posy = 3

negy = 4

posz = 5

negz = 6

NONE = -99

Global variables :

xmax, ymax, zmax, cell_width	– (real)
R_star, R_inf, V_inf, M_dot, T_eff	– (real)
nx_cell, ny_cell, nz_cell, Ngrid, n_modelgrid, n_nubin	– (ineger)
idum, debug	– (ineger)

Parameters for parallelization:

n_tasks	– (ineger)
my_rank	– (ineger)

Physical constants: ...

Parameters for testing:

freq_line=light_speed/1216.D-8, **osc_line**=0.416D0 ! for Ly_alph line
nu_min= 2.14286D15, **nu_max**=3.D15

=====
main.f90
=====

In this main subroutine we do the following:

1. Parallelization with Message Passing Interface (MPI). MPI is a language-independent communications protocol used to program parallel computers. Parallelization starts with compilation command
2. Read input CALL **read_input**(n_pack, iseed)
3. Initialing seed from the system time CALL **date_and_time**(values = TT)
4. Define length of dynamic arrays (cell, package, model_grid)
5. Define size of the grid cells which can be the same for all cell sides (cell_width) or can be different (deltax, deltay, deltaz – can be calculated also in the subroutine setup_grid)
6. Set up of the propagation grid CALL **setup_grid**()
7. Write cell()%indexc and cell()%corner into the folder **cells.dat**
8. Set up outflow (model grid) CALL **setup_model_grid**() (optionally we can call CALL **toy_model**())
9. Update model grid properties (model will be updated after consistance temperature calculation from teh radiation field) CALL **update_grid**()
10. Write cell()%corner, model_grid(cell()%model_index)%rho into the folder **modelgrid.dat**
Write model_grid(cell(L)%model_index)%rho into the folder **density.dat**
11. Initialization of photon packages from the photosphere CALL **init_photsphere**(n_pack)
12. Propagation of the photon in 3D grid CALL **propagation**(n_pack, opa_cell, lower_opa, delta_opa)
13. Updating of photon packages CALL **update_packages**(n_pack)
14. Calculate the spectrum CALL **do_spectrum**(n_pack)

Ngrid	- total number of grid cells ($N_{\text{grid}} = n_{x_cell} * n_{y_cell} * n_{z_cell}$)
Nmax	- max number of grid cells (it is defined at the beginning)
<code>cell(Ngrid)</code>	- dynamic array cell
<code>package(n_pack+1)</code>	- dynamic array package
<code>model_grid(n_modelgrid+1)</code>	- dynamic array for model grid structure
<code>cell_width</code>	- size of the grid cells in case of regular grid (but it can be different in x, y, and z direction)

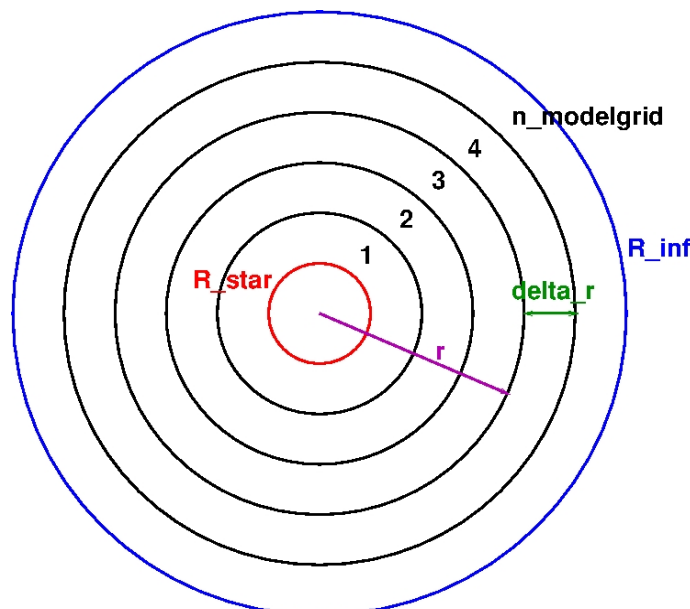
===== setup_grid.f90 =====

Set up propagation grid:

1. Initialize indexes of each cell in x, y, and z direction
`cell() %indexc(1), cell() %indexc(2), cell() %indexc(3)`
2. Initialize size of each grid cells in x, y, and z direction in case that grid cell sides are not the same; `cell() %deltax, cell() %deltay, cell() %deltaz`
3. Initialize coordinates of the lower left corner of each cell
`cell() %corner(1), cell() %corner(2), cell() %corner(3)`

===== toy_model.f90 =====

1. Define the size of the model grid, $\Delta r = (R_{\text{inf}} - R_{\text{star}}) / n_{\text{modelgrid}}$; for 1D Δr is the radial distance between two shells
2. Define:
 - a) `model_grid(n_modelgrid) %rwind = r = R_star + I * delta_r`, $I=1, n_{\text{modelgrid}}$ - the radius inside the wind
 - b) `model_grid(I) %vel = (r/R_inf)*V_inf` - velocity of the i-th grid wind model
 - c) `model_grid(I) %rho = M_dot / (4.D0 * pi * r**2 * model_grid(I) %vel)` - density of the i-th grid wind model
3. Define which model grid cell corresponds to the propagation grid cell



Instead of to_model.f90 we now use more realistic model.

```
=====
create_model_data.f90
=====
```

This program create the model grid data. Make sure that everything is in cgs units, it means either to transfer everything in units of solar or stellar radius. All data in this program should be consistence with data in subroutine read_1D_model.f90.

Model grid for now is 1D, but it should be possible also to create 2D or 3D. For now we set all values manually but it can be replaced by real HD data.

I – model grid cell index ($I=1, n_modelgrid$)

rwind(I) – radius of the wind

vel(I) – velocity of the wind in the given shell radius

rho(I) – density of the wind in the given shell radius

T(I) – temperature of the wind in the given shell radius

mass_frac (L) – list of the element abundances (or mass-fractions) of the wind in the given shell radius, $L=1, n_elements$ (for now we put 26 elements from H to Fe but assuming that all wind has only H abundance equal 1.D0 and all other element abundances are set to 0.D0). Commented lines in the program later can be uset to set solar abundances of given elements.

Mass fraction data may be provide either from the consistence RHD calculation or we can derived it from observation (making comparison between synthetic spectrum and real observation).

```
=====
model_data.dat
=====
```

T_eff - effective temperature of star (40000.)

R_star - radius of star in units of stellar radius (1) (1.)

R_inf - outer radius of the wind in units of stellar radius (r where $V = V_inf$)

V_inf - terminal velocity of the wind (3000.)

M_dot - mass-loss rates (1.E-14)

n_modelgrid - number of model grids (related to the outflow) – for 1D it is related to number of radial shells; for 3D model grid will be identical as propagation grid (200)

Data below written set of data are given in same order as it written in cerate_model_data.f90.

```
=====
setup_model_grid.f90
=====
```

Depending of the model_type in this subroutine we read 1D, 2D or 3D wind data (outflow). For now we made subroutine for reading 1D model but it would be good to make additional subroutin for reading 2D and 3D data.

```
=====
read_1D_model.f90
```

Similar procedure as in toy_model.f90 (see there).

```
=====
update_grid.f90
```

Calculate ground level population and total population of every every ion of particular element in the given model grid cell. For this purpose we called fw subroutines: find_e_nd, ionisation fraction and part_fun.

Calculate electron number density, population number of the ground level and total population number for every model grid cell for given composition and corresponding ionization stages.

```
=====
find_e_nd.f90
```

Calculation of electron number density iteratively in model grid cellmodel_grid_index

model_grid_index – input

el_nd -output

minc – very small number which is use to reach convergency

$x_{(n+1)} = x_n - (x_n - x_{(n-1)}) / (f(x_n) - f(x_{(n-1)})) * f(x_n)$ – iteration function

$x_n = el_nd_1$ - Smallest value for the el_nd (can be either 1. or 0.)

$x_{(n-1)} = el_nd_2$ - the largest value for the el_nd and I tiis when we assume that all H is ionized

$f(x_{(n-1)}) = func1(el_nd_1)$

$f(x_n) = func2(el_nd_2)$

CALL **f_edens(model_grid_index, el_nd_1, func1)**

CALL **f_edens(model_grid_index, el_nd_2, func2)**

diff = $(el_nd_2 - el_nd_1) / (func2 - func1) * func2$

```
=====
f_edens.f90
```

Function to calculate the root of the electron number density

model_grid_index, el_nd – input

func – output

indexe = 1, n_elements

indexi = 1, numb_ions (=elements(indexe)%nions)

temp = model_grid(model_grid_index)%T

abund = model_grid(model_grid_index)%grid_comp(indexe)%abund

rho = model_grid(model_grid_index)%rho

atom_mass = elements(indexe)%atom_mass)

$$\mathbf{func} = \rho * \sum_{\text{indexe}} (\text{abund} / \text{atom_mass}) * \left(\sum_{\text{indexi}} (\text{indexi} - 1) * \text{frac}(\text{indexe}, \text{indexi}, \text{temp}, \text{el_nd}) \right)$$

frac followe from CALL **ionization_fraction(indexe, indexi, temp, el_nd, frac)**

=====
ionization_fraction.f90
 =====

Calculate the fraction of atoms of chemical element indexe in ionization stage indexi relative to the total number of atoms of that element.

indexe, indexi, temp, el_nd – input

frac - output

numb_ions = elements(indexe)%nions

$$\text{frac} = \frac{\sum_{\text{indexi}=1}^{\text{numb_ions}-1} \text{el_nd} * \text{sb_factor}(\text{indexe}, \text{I}, \text{temp})}{\sum_{\text{m}=1}^{\text{numb_ions}} \prod_{\text{l}=\text{m}}^{\text{numb_ions}-1} (\text{el_nd} * \text{sb_factor}(\text{indexe}, \text{I}, \text{temp}))}$$

CALL **saha_boltzmann_factor(indexe, I, temp, sb_factor)**

=====
saha_boltzmann_factor.f90
 =====

Calculate the ratio sb_factor between population of any two excitation levels indexi and (indexi+1)

indexe, indexi, temp – input

sb_factor - output

CALL **part_fun(indexe, indexi, temp, U1)**

CALL part_fun(indexe, indexi, temp, U2)

$$\text{sb_factor} = U1 / U2 * \text{saha_const} * \text{temp}^{**}(-3/2) * \text{EXP}(\text{elements}(\text{indexe})\% \text{ions}(\text{indexi}) \% \text{ion_potential} / (\text{BOLK} * \text{temp}))$$

=====
part_fun.f90
 =====

indexe, indexi, temp – input

U -output

g_level = elements(indexe)%ions(indexi)%levels(indexl)%stat_weight - stat. weight of ground level

e_level = elements(indexe)%ions(indexi)%levels(indexl)%exci_energy – Excitation energy of the excited level indexl

nlevels = elements(indexe)%ions(indexi)%nlevels

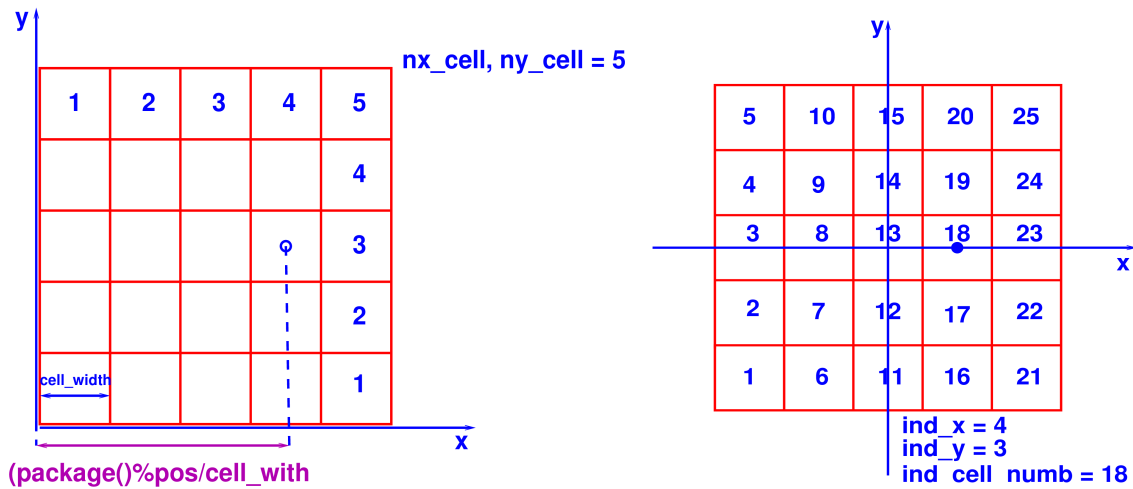
indexl = 2, nlevels

$$U = \sum g_level * \text{EXP}(-(e_level - e_gl) / \text{BOLK} / \text{temp})$$

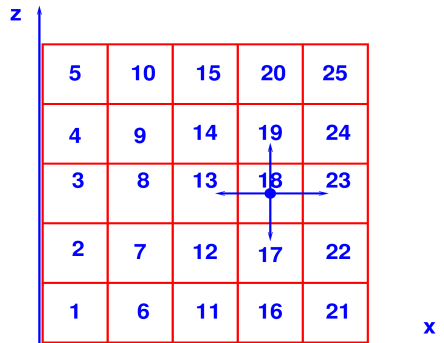
$e_{\text{level}} - e_{\text{gl}}$ - Excitation energy relative to the ground level (state)

=====
init_photosphere.f90
 =====

1. Define luminosity of the star, $L_{\text{star}} = 4.D0 \cdot \pi \cdot (R_{\text{star}})^2 \cdot \sigma \cdot T_{\text{eff}}^4$
2. Place photon on the photosphere's surface
 CALL **random_unitvector1**(direction, sint, cost, sinp, cosp)
package(I)%pos = $R_{\text{star}} \cdot \text{direction}$, $I = 1$, n_{pack}
3. Define random direction of the photon package outward from the photosphere
 CALL **random_unitvector2**(directionnn)
package(I)%dir = direction
4. Put the photon to the corresponding grid cell; determine the propagation grid cell index where is the photon, **package(I)%cell_numb** = ind_cell_numb
5. Flag the packet as an active (**package(I)%active**=1) r-pkt (**package(I)%typ** = type_rpktand) allow all kind of boundary cell crossings (**package(I)%last_cross** = NONE)
6. Assign rf energy (**package(I)%e_rf** = $L_{\text{star}}/n_{\text{pack}}$) and frequency (**package(I)%freq_rf** = freq) to the packet, CALL **freq_from_planck**(freq)
7. Convert the energy and frequency to their cmf values, CALL **doppler_factor**(I, D)
package(I)%freq_cmf = $\text{package(I)\%freq_rf} \cdot D$
package(I)%e_cmf = $\text{package(I)\%e_rf} \cdot D$



$\text{ind_x} = (\text{package()} \% \text{pos} / \text{cell_width} + \text{nx_cell} / 2) + 1$



$\text{nc} = \text{package}(\text{pack_ind}) \% \text{cell_numb} = 18$
 $\text{next_cell}(\text{xpos}) = \text{nc} + \text{ny_cell} \cdot \text{nz_cell} - \text{movement in x direction to index 23}$
 $\text{next_cell}(\text{xneg}) = \text{nc} - \text{ny_cell} \cdot \text{nz_cell} - \text{movement in x direction to index 13}$
 $\text{next_cell}(\text{zpos}) = \text{nc} + 1 - \text{movement in z direction to index 19}$
 $\text{next_cell}(\text{zneg}) = \text{nc} - 1 - \text{movement in z direction to index 17}$

=====

update_packages.f90

=====

In this subroutine photon packages are updated according what happened with them i.e. which kind of processes happened.

1. If the packet is of type rpkt, it represents a photon. So it needs to be propagated.
CALL **do_rpackage**(pack_index)
2. If the packet is of type kpkt, it represents thermal kinetic energy. Sample all possible cooling processes and randomly select one of them
CALL **do_kpackage**(pack_index)
3. If the packet is of type ipkt, it represents atomic internal energy (excitation/ionization). Calculate all transition probabilities and randomly select one of them (macro-atom formalism)
CALL **do_ipackage**(pack_index)

=====

do_rpackage.f90

=====

(!!!!!!Propagation of the photon in 3D grid!!!!!!)

We do atomic physics not propagate packets.

Packet of all types (for now we have only photons) we need to update depending on what they are e.g. do_rpackage.f90 treats the photons propagation

1. Calculate the shortest distance to the cell surface which photon will cross and return this distance (**dist**), the cell the photon will go to (next_cell, -99 in case of photon leaving the simulation grid(volume)), and make the photon remember which surface will cross (update package(pack_index)%last_cross),
CALL **boundary**(pack_index, cell_dist, next_cell)
2. Calculate distance the photon needs to travel to come to resonance with the next line (**e_dist**). This assumes homologous expansion i.e. velocity is proportional to r. Projected gradient of the projected velocity in a direction of the photon propagation is more complicated in case of no homologous expansion,
CALL **event_dist**(pack_index, cell_dist, e_dist, event)
3. If e_dist < cell_dist then:
 - a) move photon package from the current position for some distance,
CALL **move_package**(pack_index, e_dist)
 - b) determine which kind of event will happen with r package (photon)
CALL **do_rpackage_event**(pack_index, event)
4. If e_dist > cell_dist then:
 - a) Move package from the current position for the cell_dist
CALL **move_package**(pack_index, cell_dist)
 - b) If package escaped the calculation volume (next_cell=-99) then it become no-active and package type is update to the type_escaped, else the cell number is updated
CALL **change_cell**(pack_index, next_cell)

boundary.f90

1. Calculate the distances to the all cell surfaces from the photon current position along the ray and print them (formula for this can be found in http://www.roe.ac.uk/ifa/postgrad/pedagogy/2009_forgan.pdf Fig. 2, An Introduction to Monte Carlo Radiative Transfer, Duncan Forgan)
dist_plusx, dist_minx, dist_plusy, dist_miny, dist_plusz, dist_minz
2. Initializations - large distance bigger than grid size (all calculated idstances should be shorter), **dist** = 1.D99
3. Remember which surface the photon crossed in its last movement (positive or negative direction and which surface), **forbidden** = package(pack_index)%last_cross
4. Find the shortest of above distances
5. Printing

last_cross parameter remembers which boundary was crossed last. After an emission this memory is erased = NONE (last_cross = NONE – allowed to cross any cell boundary).

If the packet crosses a boundary, then packet remembers which boundary it crossed (posx, posy, posz, negx, negy, negz). This to make sure that packets do not scatter among grid cells.

event_dist.f90

1. Randomly choose optical depth, **tau_rand**
2. Initialize optical depth and distance , **tau** = 0.D0 , **dist** = 0.D0
3. Calculate the population number i.e. number density of hydrogen. Assuming that all hydrogen is at the ground state we can use number density of hydrogen as its population number, **pop_number** = model_grid(get_package_model_index(pack_index))%rho / mp_g
FUNCTION **get_package_model_index**(pack_index) - For the pack_index calculate the model_index
4. For now we neglect cont. opacities, but the routine was written generally to allow for adding cont. opacity in the future
5. Calculate the opacity in co-moving frame and transform to the lab frame . According to Mihalas and Mihalas Eq. 90.8 this is achieved by CALL **doppler_factor**(pack_index, D)
kappa_cont = D * kappa_cont
6. If package(pack_index)%freq_cmf < freq_line then
 - a) Calculate distance the photon needs to travel to come to resonance with the next line. This assumes homologous expansion i.e. velocity is proportional to r. Projected gradient of the projected velocity in a direction of the photon propagation is more complicated in case of no homologous expansion, **l_dist**
 - b) Calculate optical depth in the next line (Sobolev, dv/dr dependent) and continuum optical depth accumulated up to the line
CALL **move_package**(dummyspackage, l_dist)
CALL **velo**(dummyspackage, vel_vec) => **pop_number, tau_line, tau_cont**
7. Do a step by step analysis of which event occurs and return the distance and corresponding event
8. Use **dummyspackage** to move to the l_dist only to check whether or which kind of interaction will happen at l_dist

Check: **package(pack_index)%freq_cmf > freq_line** then

8.1. Check1: If **(tau_rand - tau) > tau_cont** then

8.1.1. Check2: If **(tau_rand - tau) > (tau_cont + tau_line)** then no interaction

a) calculate optical depth **tau = tau + tau_cont + tau_line**

b) calculate distance **dist = dist + l_dist**

c) Check3: If **dist > cell_dist** then the package propagates to the next cell

e_dist = cell_dist + 1.D20

do_loop = 0

event = rpkt_eventtype_changecell

8.1.2. Check2: If **(tau_rand - tau) < (tau_cont + tau_line)** then the line interaction happens

e_dist = dist + l_dist

do_loop = 0

event = rpkt_eventtype_lineinteraction

8.2. Check1: If **(tau_rand - tau) > (tau_cont + tau_line)** then continuum process will happen

e_dist = dist + (tau_rand - tau)/kappa_cont

do_loop = 0

event = rpkt_eventtype_continuum

Check: **package(pack_index)%freq_cmf < freq_line** the package cmf frequency is too red to interact to another line - No line interact anymore

tau_cont = kappa_cont * (cell_dist - dist)

8.1. Check4: **(tau_rand - tau) > tau_cont** then

e_dist = cell_dist + 1.D20

do_loop = 0

event = rpkt_eventtype_changecell

8.2 Check4: **(tau_rand - tau) < tau_cont** then continuum absorption happens

e_dist = dist + (tau_rand - tau)/kappa_cont

do_loop = 0

event = rpkt_eventtype_continuum

=====
do_rpackage_event.f90
=====

1. If **event = rpkt_eventtype_lineinteraction**

In this case the package interacts with a line. In the general case we now would make it an i-package and assign the i-package to the upper level of the corresponding bound-bound transition . However, for now we restrict ourselves to the pure scattering case, i.e. we immediately re-emit the package isotropically in the cmf and conserve the cmf frequency

print*, freq_line, package(pack_index)%freq_cmf

CALL **emit_rpackage(pack_index)**

2. If **event = rpkt_eventtype_continuum**

In this case the package undergoes a continuum event. In the general case we need to decide now if this was a electron-scattering, free-free or bound-free absorption (sample the different opacities) and act accordingly. In case of e/s the packet stays and an rpkt and is re-emitted isotropically in the cmf. For ff it becomes a kpkt, In the case of bf we have to check further if it will go to a kpkt or ipkt (bf contribute to both the thermal kinetic and internal energy pools). For now we set the electron number density to zero, so no continuum opacity should

be there print*, 'Continuum event occurred. This should not happen for now!'

As an easy next step make sure that a proper e/s opacity is calculated, than the following lines should work and give isotropic re-emission in the cmf

CALL **emit_rpackage(pack_index)**

=====

move_package.f90

=====

1. Move photon package from the curent position for some distance,
update **package(pack_index)%pos**
2. Calculate the position of package
3. Deactivate packets which travel beyond the photosphere (if position
vec_length(package(pack_index)%pos < R_star => package(pack_index)%active = 0)
FUNCTION **vec_length(vector)**
4. Rest frame quantities do not change while propagating without any events, but cmf
quantities need to be updated,
5. **package(pack_index)%freq_cmf** and **package(pack_index)%e_cmf**
6. Printing

=====

change_cell.f90

=====

Change the cell number

1. If package escaped the calculation volume (next_cell=-99) then it become no-active and
update into type_escaped, else it update the cell number
2. If next_cell < 0 then
package(pack_index)%typ = type_escaped
package(pack_index)%active = 0 else
package(pack_index)%cell_numb = next_cell

=====

do_spectrum.f90

=====

1. Set up the frequency grid to extract spectrum
spectrum(i)%freq, spectrum(i)%flux, spectrum(i)%esc
2. Loop over all packets and take all which actually escaped
3. Only bin those packets which are in the allowed frequency range
4. Write to file

TESTS

cell.dat – coordinates and indexes of the lover left corner of each cell

1column - **cell(L)%indexc(1)**

2column - **cell(L)%indexc(2)**

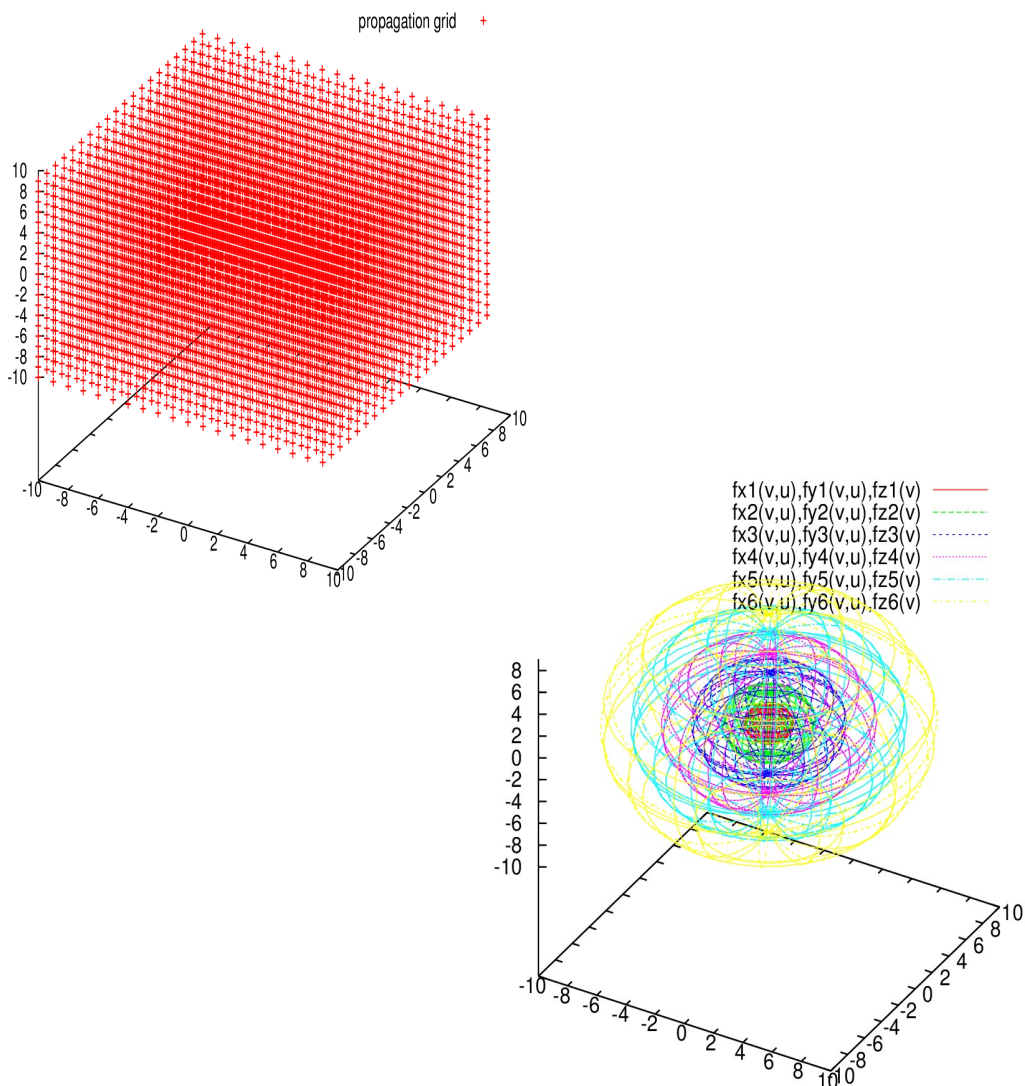
3column - **cell(L)%indexc(3)**

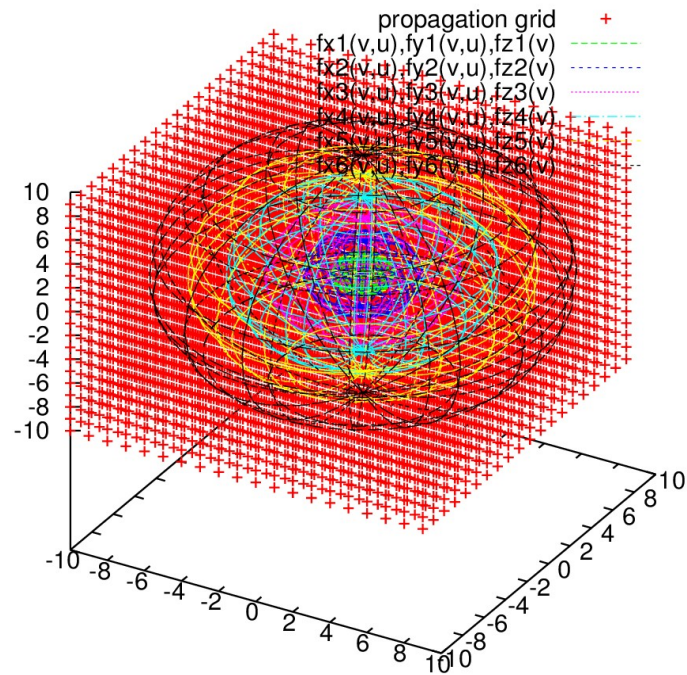
4column - **cell(L)%corner(1)**

5column - cell(L)%corner(2)

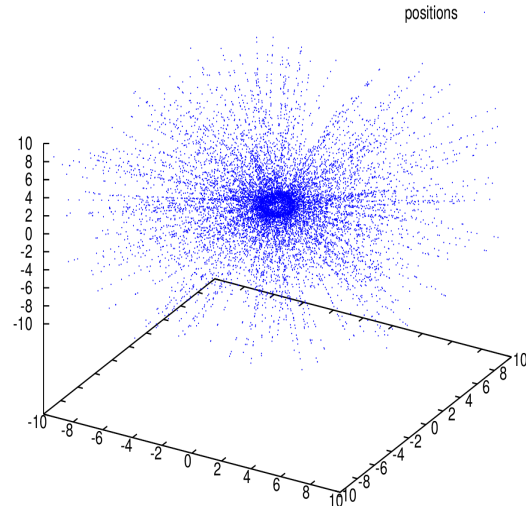
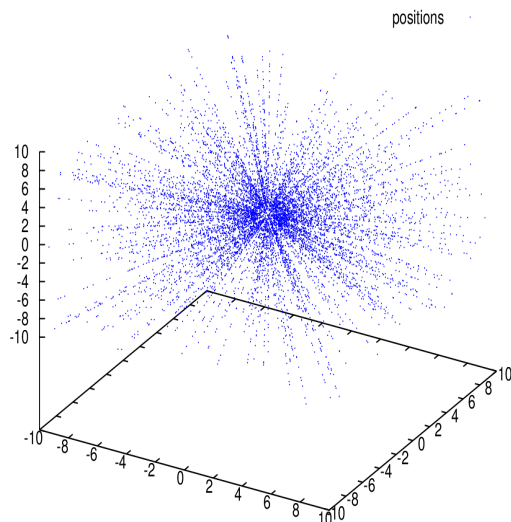
6column - cell(L)%corner(3)

1. For faster calculation we calculated 10x10x10 propagation grid with 20 cells in every direction. In main.f90 activate OPEN (UNIT=2, FILE='propa_grid_20cells.dat') and in the setup_grid.f90 activate write(2,*) cell(L)%indexc, cell(L)%corner. Results of testing are in the folder 3Dgridwind/propagation/test.
propa_grid_20cells.dat (corresponds to the cell.dat)
grid.gp
grid-prop-20cells.eps
2. We calculated model grid (outflow) for now only in 1D. This means that density and velocity of the wind are changes only in radial direction => wind is consists of radial shells. In toy_model.f90 uncomment OPEN (UNIT=7, FILE='model_grid_cells.dat') and WRITE(7,*) I, model_grid(I)%rwind
Results of testing are in the folder 3Dgridwind/propagation/test.
model_grid_cells.dat
model_grid.gp
grid-model.eps
grid-prop-model.eps



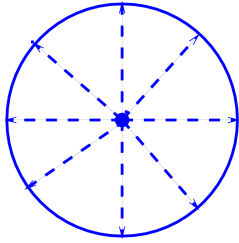


3. Packages are sent from the point source or from the surface of the sphere with radius R_{star} .
 In propagation.f90 uncomment `OPEN (UNIT=3, FILE='position.dat')` and `WRITE(3,*)`
`pack_index, package(pack_index)%cell_numb, package(pack_index)%pos`
 For the point also in main.f90 uncomment `CALL init_photonpack(n_pack, R_star)` and for
 sphere uncomment `!CALL init_photsphere(n_pack, R_star)`
`position_point.dat`
`position_sphere.dat`
`position.gp`
`position_point.eps`
`position_sphere.eps`

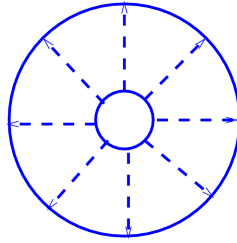


1. Calculate escape probability emitting packages from. Validate against analytic solution all without outflow just to clear propagation routines.
 - 1.1. point source
 - 1.2. sphere surface
 - 1.3. homogeneous emitting sphere with and without analytic solution available

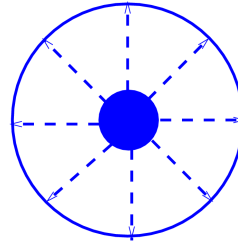
Point source



Sphere surface



Homogeneous sphere



- 1.1. Analytic and calculated solution should be the same for point source (we need finer propagation and model grid), in main.f90 uncomment **DOUBLE PRECISION**, **PARAMETER :: upper_opa=2.D0/9.D0, lower_opa=0.1D0/18.D0 !** (this opa are valid for opacity for photons sent from the photosphere $R_{\text{star}} = 2$), and **CALL init_photonpack(n_pack, R_star)**

Pure absorption test: in do_event.f90 comment all expect **package(pack_index)**

%active=0. Properly change name of the file in position.f90 when we change number of photons

escprob-point-abs-1000.dat

escprob-point-abs10000.dat

escprob-point-abs-100000.dat

esc_prob.gp

pescprob_point_abs.eps

Absorption + Scattering: In do_event.f90 use all IF statement with random number before it.

escprob-point-scat-1000.dat

escprob-point-scat-10000.dat

escprob-point-scat-100000.dat

esc_prob.gp

pescprob_point_scat.eps

For lower and upper opacity in main.f90 we choose on such way that optical depth is from 0.1 till 4.

Since $\tau = \kappa * R_{\text{wind}} \Rightarrow$

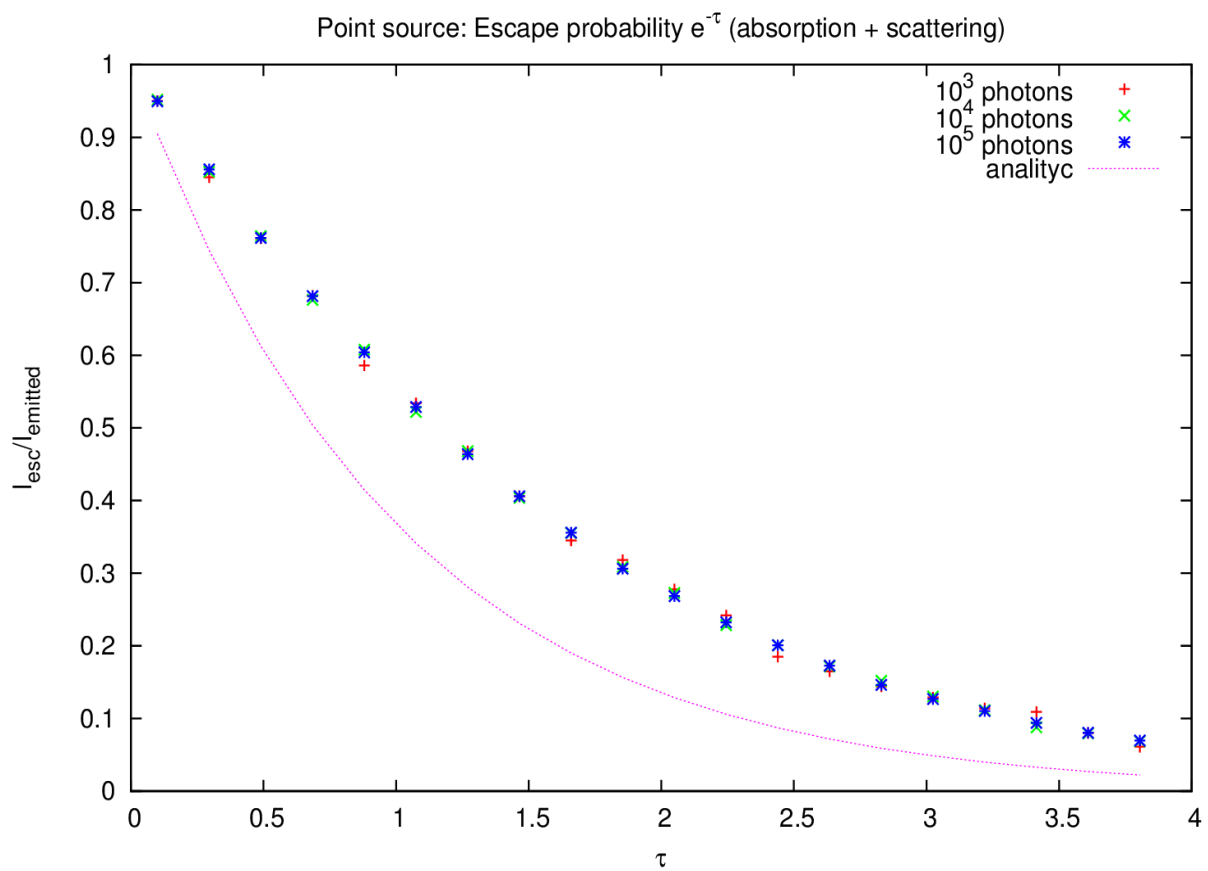
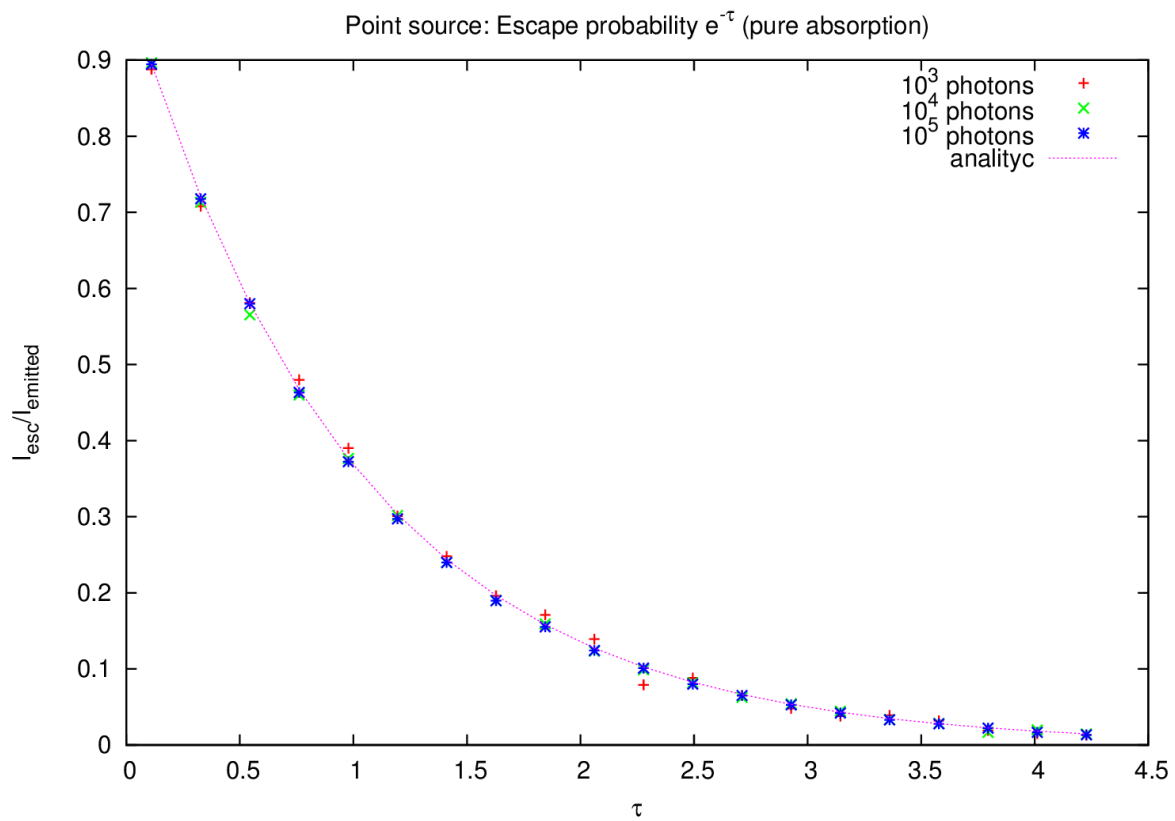
for $R_{\text{wind}} = 20$ and point source: $\kappa_{\text{lower}} = 4/20 = 0.2$; $\kappa_{\text{upper}} = 0.1/20 = 1/200$

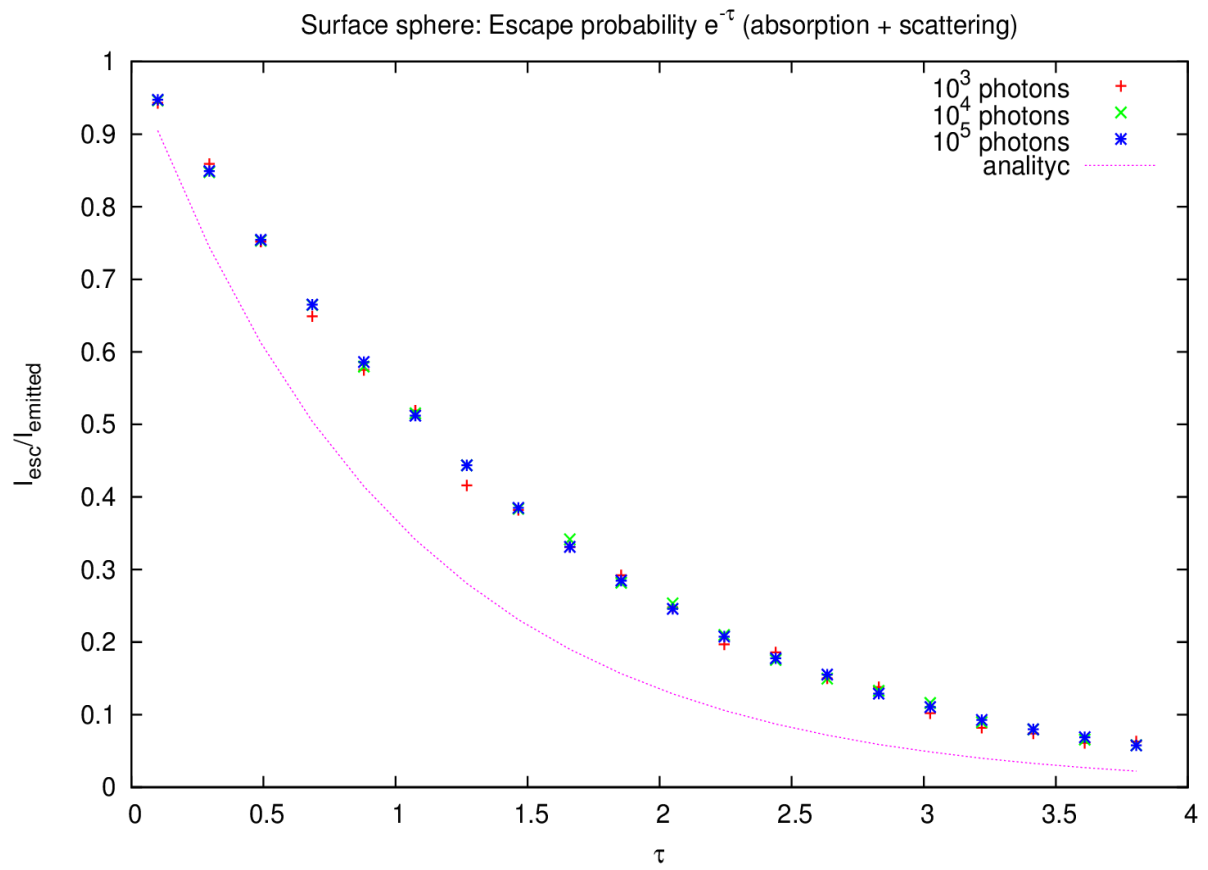
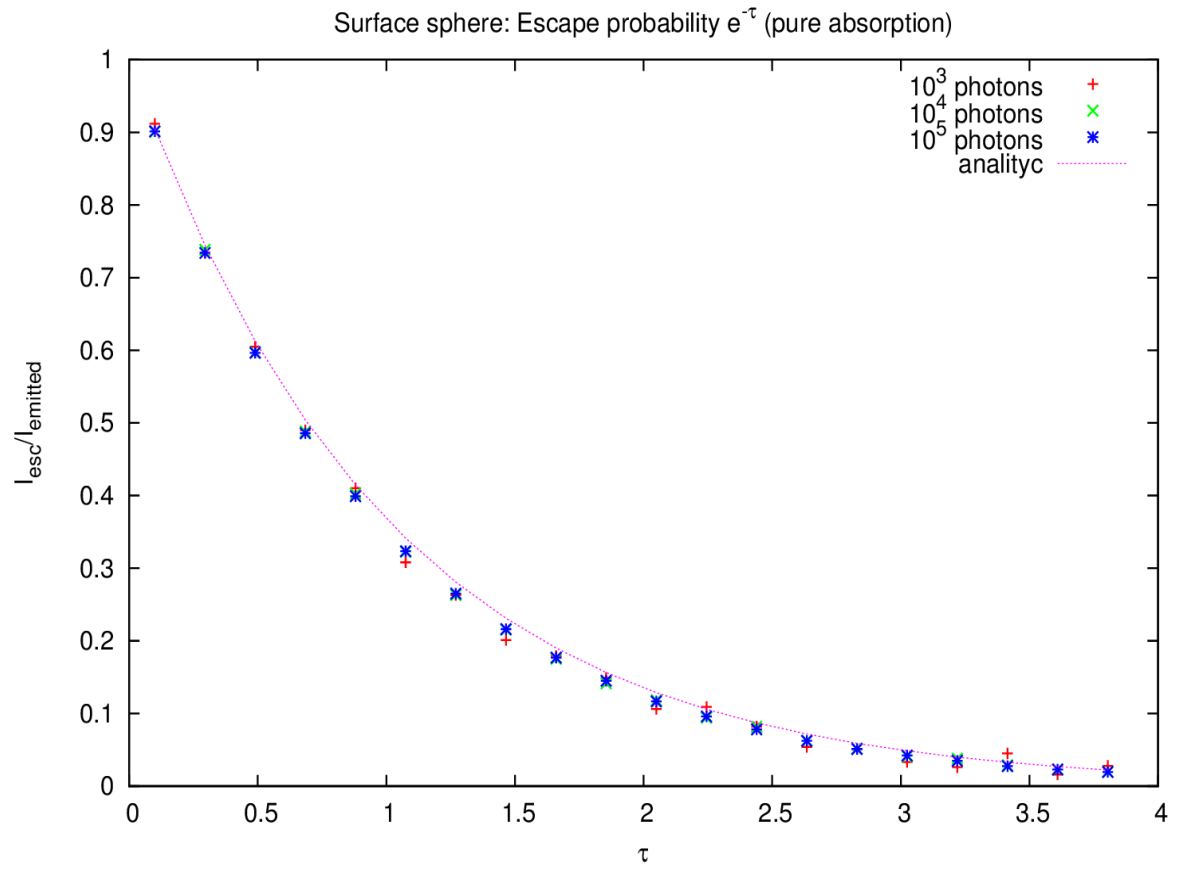
for $R_{\text{wind}} = 20$, $R_{\text{star}} = 2$ and sphere: $\kappa_{\text{lower}} = 4/(20 - 2) = 2/9$; $\kappa_{\text{upper}} = 0.1/(20-2) = 0.1/18$.

In test/propagation.f90

$\tau = \kappa * \text{distance}$ (in our case for the point source it is R_{wind})

$I = I_0 e^{(-\tau)} \Rightarrow \beta = I/I_0 = e^{(-\tau)} = \text{number of escaped package/number of emitted packages} - \text{escape probability.}$



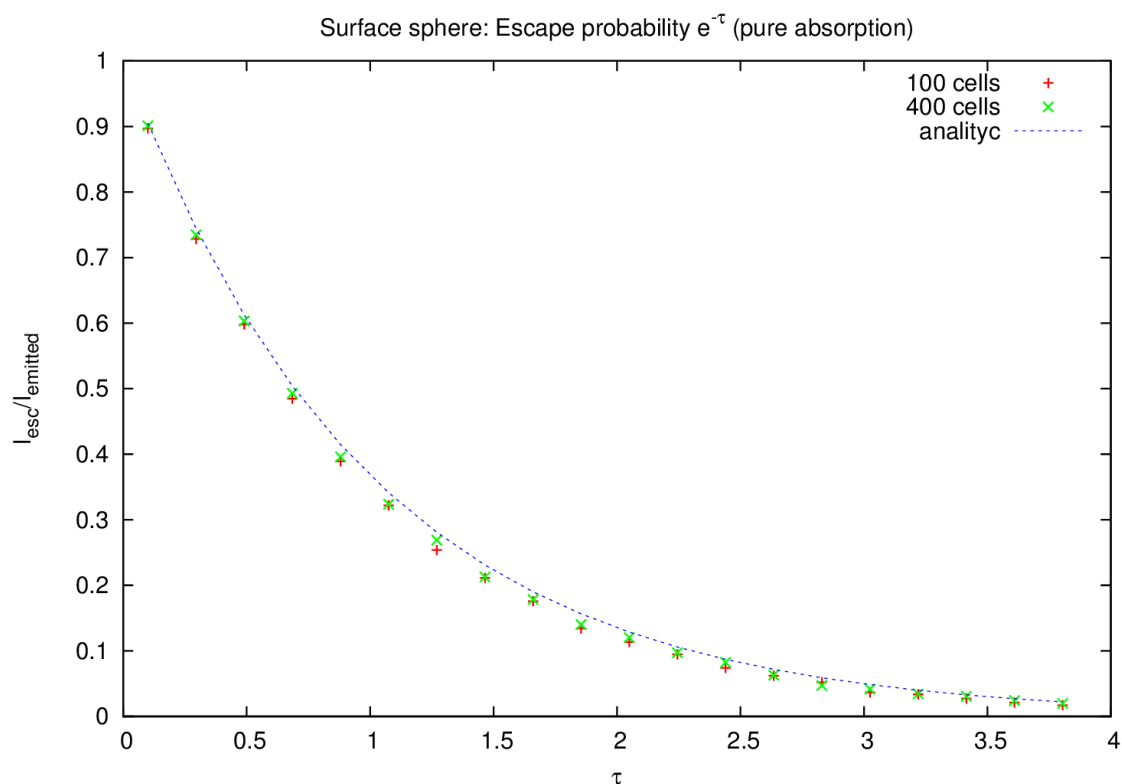
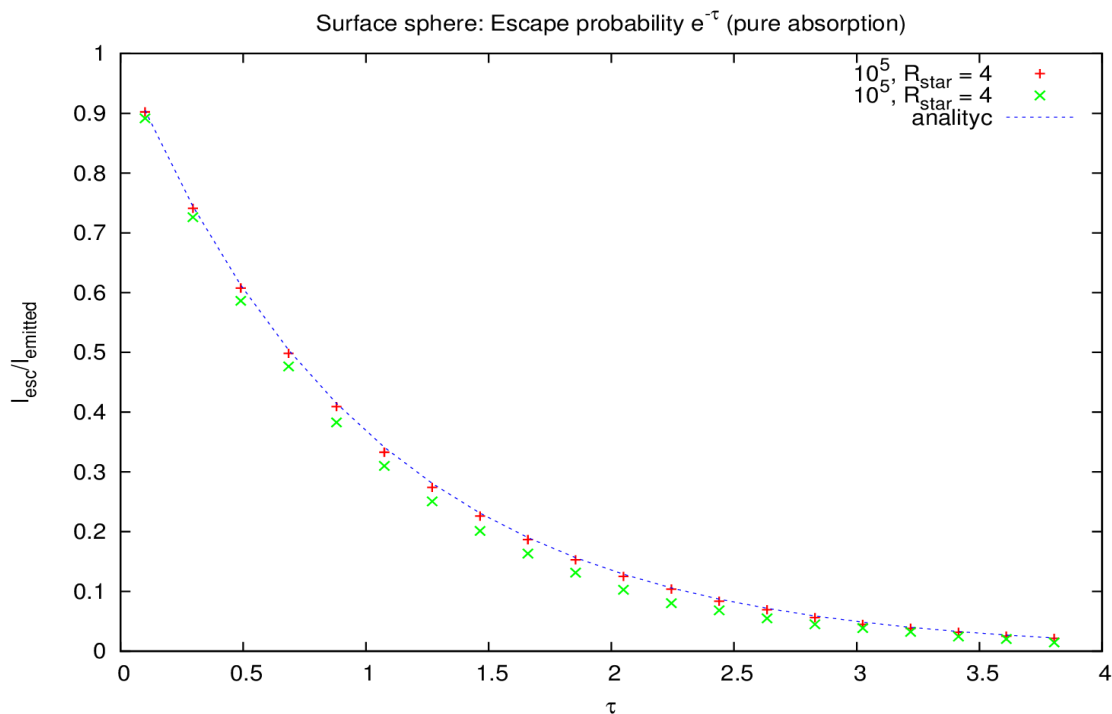


In case of pure absorption with packages emitted from surface sphere, there is discrepancy between analytic and calculated escape probability. We are still not sure why. But in the limited case (when $R_{\text{star}} \rightarrow 0$) agreement is much better (we reach point source case).

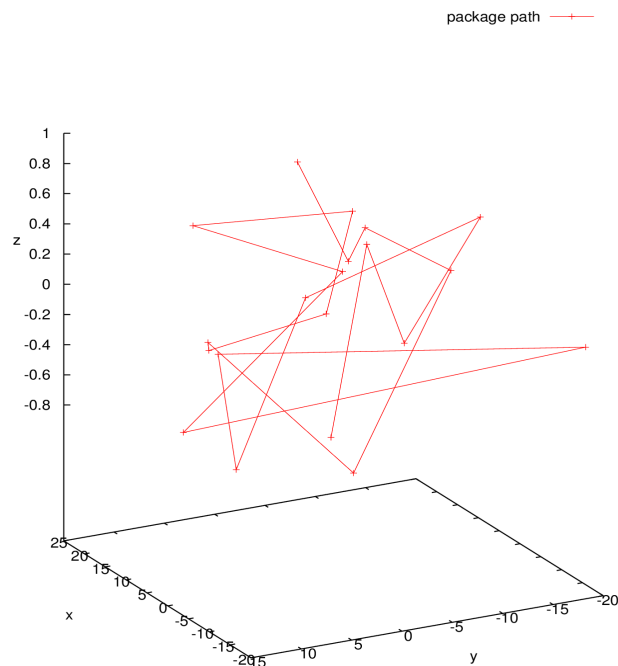
Test with $R_{\text{star}} = 0.2$ (escprob-sphere-abs-r0.2.dat) and with $R_{\text{star}} = 4$ (escprob-sphere-abs-r4.dat). escprob_sphere_scatter-radius.eps

We checked whether resolution of the propagation and model grid may influence result (escprob-sphere-abs-r0.2.dat and escprob-sphere-abs-r4.dat). But, higher resolution does not give better agreement.

escprob_sphere_scatter-resolution.eps



Propagation of the packages



position_sphere_path.dat
path.gp
path.eps

```
=====
read_composition.f90
```

```
=====
! Read the file compose_adata.dat and remember
! elements(n_elements)
! elements(I)%atom_number, I=1,n_elements
! elements(I)%nions
! elements(I)%ions(nions)
! elements(I)%ions(J)%ion_stage, J=1,nions
```

1. From compose_adata.dat read only first line which corresponds to the number of the chemical elements (n_elements) involved in the calculation
2. Allocate the memory to the
elements(n_elements)
3. Loop over all chem.elements involved i.e. continue to read all other lines in the compose_adata.dat
elements(I)%atom_number = Z, I=1,n_elements
elements(I)%nions = nions
4. Assign lowerion to the current ion which we will use to count number of ions .
This is important because we can play only with 3 and 4 ion.stage of some element
current_ion = lowerion

5. Allocate the memory to the
elements(I)%ions(nions)
6. Loop over all ions (nions) of given chem.element
elements(I)%ions(J)%ion_stage, J=1,nions

=====
read_atomic_data.f90
=====

! Read the file compose_adata.dat and remember
! elements(I)%ions(J)%ion_potential
! elements(I)%ions(J)%nlevels
! elements(I)%ions(J)%levels(n_lev)
! elements(I)%ions(J)%levels(K)%exci_energy
! elements(I)%ions(J)%levels(K)%stat_waight
! ntransitions

1. From compose_adata.dat read only first line which corespons to the number of the chemical elements (n_elements) involved in the calculation
2. Read only the first line of every element in particular ionisation stage
3. In the case that the a_nub of the next reading line is not equal to the atomic number of the last read element the last read element (donated with remember_last_element) then ionisation potential will not change (or increase), e.i. ion.pot. increment ionoffset = 0.
4. Flag for the deciding for which elements of given ionisation stage we will use data
For flag = 0. we will only read data but we will not use them, otherwise we will use them.
5. Loop over all elements (which we already read from compose_adata.dat)
6. Loop over all ions
7. If in the file atomic_data.dat we found particular element in the given atomic stage wish we have in the compose_adata.dat list, this condition will be done.
8. Associate the pot.energy i_pot (multiplie with e_v) from compose_adata.dat to the
elements(I)%ions(J)%ion_potential.
9. Associate the numb.lavel n_lev from compose_adata.dat to the
elements(I)%ions(J)%nlevels
10. Allocete the memory; define how large should be **elements(I)%ions(J)%levels (n_lev).**
11. Loop over all levels.
12. Encrease the level energy l_e (multiplie with e_v) by ionoffset i.e. for the ion.pot. of the graund level,
elements(I)%ions(J)%levels(K)%exci_energy
elements(I)%ions(J)%levels(K)%stat_waight
13. Calculate total number of transitions of the given level.
14. When the all levels of the given element in the particular ionisation stage is used, then put flag to 1. It means that we will continue to read file atomic_data.dat from the next element.
15. Increase the ionoffset by the pot.ene. i_pot of particular ion.stage . For the neutral elements ionoffset = 0.
16. Remember last element we pass.
17. Calculate the real number of transitions (how many transitions are involved)
ntransitions = ntransitions/2.
18. Test printing

For the given **compose_dat.dat**

number of elements	2			
	element_index	Z	lowerion	upperion
	1	1	1	2
	2	6	3	4

and **atomic_data.dat**

atom.numb	ion.stage	numb.lev.	ion.pot.	
1	1	22	13.5979996	
lev.numb	lev.energy	stat.weight	numb.trans	
1	0.0000000000000000	2.0000000000000000	5	
2	10.1984796524047852	2.0000000000000000	4	
3	10.1985101699829102	6.0000000000000000	9	
4	12.0871000289916992	2.0000000000000000	4	
5	12.0871000289916992	6.0000000000000000	8	
6	12.0871000289916992	10.0000000000000000	7	
7	12.7481002807617188	2.0000000000000000	4	
8	12.7481002807617188	6.0000000000000000	8	
9	12.7481002807617188	10.0000000000000000	6	
10	12.7481002807617188	14.0000000000000000	5	
11	13.0541000366210938	2.0000000000000000	4	
12	13.0541000366210938	6.0000000000000000	8	
13	13.0541000366210938	10.0000000000000000	6	
14	13.0541000366210938	14.0000000000000000	4	
15	13.0541000366210938	18.0000000000000000	3	
16	13.2202997207641602	2.0000000000000000	4	
17	13.2202997207641602	6.0000000000000000	8	
18	13.2202997207641602	10.0000000000000000	6	
19	13.2202997207641602	14.0000000000000000	4	
20	13.2202997207641602	18.0000000000000000	2	
21	13.2202997207641602	22.0000000000000000	1	
22	13.5036001205444336	1.0000000000000000	0	
1	2	1	0.0000000	
1	0.0000000000000000	1.0000000000000000	0	
6	1	5	13.6	
1	0.0000000000000000	2.0000000000000000	5	
2	10.1984796524047852	2.0000000000000000	4	
3	10.1985101699829102	6.0000000000000000	9	
4	12.0871000289916992	2.0000000000000000	4	
5	12.0871000289916992	6.0000000000000000	8	
6	2	5	15.6	
1	0.0000000000000000	2.0000000000000000	5	
2	10.1984796524047852	2.0000000000000000	4	
3	10.1985101699829102	6.0000000000000000	9	
4	12.0871000289916992	2.0000000000000000	4	

5	12.0871000289916992	6.0000000000000000	8
6	3	5	20.6
1	0.0000000000000000	2.0000000000000000	5
2	10.1984796524047852	2.0000000000000000	4
3	10.1985101699829102	6.0000000000000000	9
4	12.0871000289916992	2.0000000000000000	4
5	12.0871000289916992	6.0000000000000000	8
6	4	5	25.6
1	0.0000000000000000	2.0000000000000000	5
2	10.1984796524047852	2.0000000000000000	4
3	10.1985101699829102	6.0000000000000000	9
4	12.0871000289916992	2.0000000000000000	4
5	12.0871000289916992	6.0000000000000000	8

First PRINT*, 'atomic data =', a_numb, i_stat, n_lev, i_pot in the read_atomic_data.dat will give

	a_numb	i_stat	n_lev	i_pot
atomic data =	1	1	22	13.597999600000000
atomic data =	1	2	1	0.000000000000000
atomic data =	6	1	5	13.600000000000000
atomic data =	6	2	5	15.600000000000000
atomic data =	6	3	5	20.600000000000001
atomic data =	6	4	5	25.600000000000001

TESTING

For given 2 element in the compose_dat.dat only ionization stages from 1 to 2 of the first element and from 3 to 4 of 2 element will be leater used

PRINT*, 'element index is =', I, 'atomic number =', elements(I)%atom_number, 'ionn stages =', elements(I)%nions

element index is = 1 atomic number = 1 ionn stages = 2

PRINT*, 'ionisation index =', J, 'ionisation stage =', elements(I)%ions(J)%ion_stage, &
'ionisation pot. =', elements(I)%ions(J)%ion_potential/e_v, &
'number of levels =', elements(I)%ions(J)%nlevels

ionisation index = 1 ionisation stage = 1 ionisation pot. = 13.597999600000000
number of levels = 22

PRINT*, K, elements(I)%ions(J)%levels(K)%exci_energy, elements(I)%ions(J)%levels(K)
%stat_waight
K = n_levels

First element (H) with 2 ion stages

element index is = 1 atomic number = 1 ionn stages = 2

First element (H) in ion stage 1

ionisation index = 1 ionisation stage = 1 ionisation pot. = 13.597999600000000
number of levels = 22

For the neutral elements we do not change their level energies, we just read them as they are in input file (same color)

1	0.0000000000000000	2.0000000000000000
2	10.198479652404785	2.0000000000000000
3	10.198510169982910	6.0000000000000000
4	12.087100028991699	2.0000000000000000
5	12.087100028991699	6.0000000000000000
6	12.087100028991699	10.0000000000000000
7	12.748100280761719	2.0000000000000000
8	12.748100280761719	6.0000000000000000
9	12.748100280761719	10.0000000000000000
10	12.748100280761719	14.0000000000000000
11	13.054100036621094	2.0000000000000000
12	13.054100036621094	6.0000000000000000
13	13.054100036621094	10.0000000000000000
14	13.054100036621094	14.0000000000000000
15	13.054100036621094	18.0000000000000000
16	13.220299720764160	2.0000000000000000
17	13.220299720764160	6.0000000000000000
18	13.220299720764160	10.0000000000000000
19	13.220299720764160	14.0000000000000000
20	13.220299720764160	18.0000000000000000
21	13.220299720764160	22.0000000000000000
22	13.503600120544434	1.0000000000000000

First element (H) in ion stage 1

ionisation index = 2 ionisation stage = 2 ionisation pot. = 0.0000000000000000
number of levels = 1

For ionized stages of the element, level energies are changed

ion.pot. of neutral ele. 13.6 + ion.pot of second ion.stag 0.0 = 13.597999600000000

1	13.597999600000000	1.0000000000000000
---	--------------------	--------------------

Second element (O) with 2 ion stages

element index is = 2 atomic number = 6 ionn stages = 2

First element (O) in ion stage 3

ionisation index = 1 ionisation stage = 3 ionisation pot. = 20.600000000000001
number of levels = 5

For ionization stages of the element we change the energy levels:

pot.ene. of neutral ele. 13.6 + pot.ene. of first ion.stag 15.6 + pot.ene. of second ion.stag 0.0 = 29.199999999999999

1	29.199999999999999	2.0000000000000000
2	39.398479652404788	2.0000000000000000
3	39.398510169982913	6.0000000000000000
4	41.287100028991702	2.0000000000000000
5	41.287100028991702	6.0000000000000000

First element (O) in ion stage 4

ionisation index = 2 ionisation stage = 4 ionisation pot. = 25.600000000000001
number of levels = 5

pot.ene. of neutral ele. 13.6 + pot.ene. of first ion.stag 15.6 + pot.ene. of second ion.stag 20.6 +
ene.level 0.0 = 49.799999999999997

1	49.799999999999997	2.0000000000000000
2	59.998479652404782	2.0000000000000000
3	59.998510169982907	6.0000000000000000
4	61.887100028991696	2.0000000000000000
5	61.887100028991696	6.0000000000000000

stop

=====

read_transitions.f90

=====

transitions.dat

Ionisation stage is given in astronomical standard (1 means neutral element, 2 means one times ionized element and so on).

atom.numb.	ion.stat	numb.trans of specific ion	
1	1	55	
line.numb	low.lev	up.lev	A_ij col.stren.
1	1	3	6.265000000000000E+08 -1.0000000E+00
2	1	5	1.672600000000000E+08 -1.0000000E+00
3	1	8	6.818800000000000E+07 -1.0000000E+00
4	1	12	3.437600000000000E+07 -1.0000000E+00
5	1	17	1.972900000000000E+07 -1.0000000E+00
6	2	5	2.244900000000000E+07 -1.0000000E+00
7	3	6	6.465200000000000E+07 -1.0000000E+00
8	3	4	6.313700000000000E+06 -1.0000000E+00
9	2	8	9.668300000000000E+06 -1.0000000E+00
10	3	9	2.062600000000000E+07 -1.0000000E+00
11	3	7	2.578200000000000E+06 -1.0000000E+00
12	2	12	4.948400000000000E+06 -1.0000000E+00
13	3	13	9.425600000000000E+06 -1.0000000E+00
14	3	11	1.288600000000000E+06 -1.0000000E+00
15	2	17	2.858400000000000E+06 -1.0000000E+00
16	3	18	5.145100000000000E+06 -1.0000000E+00
17	3	16	7.350100000000000E+05 -1.0000000E+00
18	4	8	3.065200000000000E+06 -1.0000000E+00
19	5	9	7.037800000000000E+06 -1.0000000E+00
20	5	7	1.835500000000000E+06 -1.0000000E+00
21	6	10	1.378800000000000E+07 -1.0000000E+00
22	6	8	3.475400000000000E+05 -1.0000000E+00
23	4	12	1.637800000000000E+06 -1.0000000E+00
24	5	13	3.391500000000000E+06 -1.0000000E+00
25	5	11	9.047100000000000E+05 -1.0000000E+00
26	6	14	4.542200000000000E+06 -1.0000000E+00

27	6	12	1.4954000000000000E+05	-1.0000000E+00
28	4	17	9.5511000000000000E+05	-1.0000000E+00
29	5	18	1.8778000000000000E+06	-1.0000000E+00
30	5	16	5.0718000000000000E+05	-1.0000000E+00
31	6	19	2.1461000000000000E+06	-1.0000000E+00
32	6	17	7.8242000000000000E+04	-1.0000000E+00
33	7	12	7.3720000000000000E+05	-1.0000000E+00
34	8	13	1.4858000000000000E+06	-1.0000000E+00
35	8	11	6.4509000000000000E+05	-1.0000000E+00
36	9	14	2.5845000000000000E+06	-1.0000000E+00
37	9	12	1.8846000000000000E+05	-1.0000000E+00
38	10	15	4.2543000000000000E+06	-1.0000000E+00
39	10	13	5.0478000000000000E+04	-1.0000000E+00
40	7	17	4.4563000000000000E+05	-1.0000000E+00
41	8	18	8.6221000000000000E+05	-1.0000000E+00
42	8	16	3.5825000000000000E+05	-1.0000000E+00
43	9	19	1.2870000000000000E+06	-1.0000000E+00
44	9	17	9.4170000000000000E+04	-1.0000000E+00
45	10	20	1.3728000000000000E+06	-1.0000000E+00
46	10	18	2.1450000000000000E+04	-1.0000000E+00
47	11	17	2.4296000000000000E+05	-1.0000000E+00
48	12	18	4.4950000000000000E+05	-1.0000000E+00
49	12	16	2.6818000000000000E+05	-1.0000000E+00
50	13	19	7.2327000000000000E+05	-1.0000000E+00
51	13	17	9.5935000000000000E+04	-1.0000000E+00
52	14	20	1.1057000000000000E+06	-1.0000000E+00
53	14	18	3.9080000000000000E+04	-1.0000000E+00
54	15	21	1.6449000000000000E+06	-1.0000000E+00
55	15	19	1.1373000000000000E+04	-1.0000000E+00

1 2 0

Read the filetransitions.dat and remember

linelist(L)%atom_number

linelist(L)%ion

linelist(L)%lower

linelist(L)%upper

linelist(L)%freq

linelist(L)%A_ul

linelist(L)%f_ul

Similar procedure as in read_atomic_data.f90. We will read only data which corresponds to the elements and their ions given in the compose_adata.dat.

As a test number of transitions has to be same a L+1 for one element

ntransitions = L+1

For more elements number of transitions will increase depending of number of lines involved

Testing

PRINT*, 'transition data =', a_num, i_stat, n_tran

PRINT*, kindex, low_level, up_level, A, col_str
 PRINT*, 'check lenght of line list', L-1, ntransitions

transition data =	1	1	55	
1	1	3	62650000.00000000	-1.0000000000000000
2	1	5	16726000.00000000	-1.0000000000000000
3	1	8	68188000.00000000	-1.0000000000000000
4	1	12	34376000.00000000	-1.0000000000000000
5	1	17	19729000.00000000	-1.0000000000000000
6	2	5	22449000.00000000	-1.0000000000000000
7	3	6	64652000.00000000	-1.0000000000000000
8	3	4	6313700.000000000	-1.0000000000000000
9	2	8	9668300.000000000	-1.0000000000000000
10	3	9	20626000.00000000	-1.0000000000000000
11	3	7	2578200.000000000	-1.0000000000000000
12	2	12	4948400.000000000	-1.0000000000000000
13	3	13	9425600.000000000	-1.0000000000000000
14	3	11	1288600.000000000	-1.0000000000000000
15	2	17	2858400.000000000	-1.0000000000000000
16	3	18	5145100.000000000	-1.0000000000000000
17	3	16	735010.0000000000	-1.0000000000000000
18	4	8	3065200.000000000	-1.0000000000000000
19	5	9	7037800.000000000	-1.0000000000000000
20	5	7	1835500.000000000	-1.0000000000000000
21	6	10	13788000.00000000	-1.0000000000000000
22	6	8	347540.0000000000	-1.0000000000000000
23	4	12	1637800.000000000	-1.0000000000000000
24	5	13	3391500.000000000	-1.0000000000000000
25	5	11	904710.0000000000	-1.0000000000000000
26	6	14	4542200.000000000	-1.0000000000000000
27	6	12	149540.0000000000	-1.0000000000000000
28	4	17	955110.0000000000	-1.0000000000000000
29	5	18	1877800.000000000	-1.0000000000000000
30	5	16	507180.0000000000	-1.0000000000000000
31	6	19	2146100.000000000	-1.0000000000000000
32	6	17	78242.00000000000	-1.0000000000000000
33	7	12	737200.0000000000	-1.0000000000000000
34	8	13	1485800.000000000	-1.0000000000000000
35	8	11	645090.0000000000	-1.0000000000000000
36	9	14	2584500.000000000	-1.0000000000000000
37	9	12	188460.0000000000	-1.0000000000000000
38	10	15	4254300.000000000	-1.0000000000000000
39	10	13	50478.00000000000	-1.0000000000000000
40	7	17	445630.0000000000	-1.0000000000000000
41	8	18	862210.0000000000	-1.0000000000000000
42	8	16	358250.0000000000	-1.0000000000000000
43	9	19	1287000.000000000	-1.0000000000000000
44	9	17	94170.00000000000	-1.0000000000000000
45	10	20	1372800.000000000	-1.0000000000000000
46	10	18	21450.00000000000	-1.0000000000000000

47	11	17	242960.00000000000	-1.0000000000000000
48	12	18	449500.00000000000	-1.0000000000000000
49	12	16	268180.00000000000	-1.0000000000000000
50	13	19	723270.00000000000	-1.0000000000000000
51	13	17	95935.00000000000	-1.0000000000000000
52	14	20	1105700.0000000000	-1.0000000000000000
53	14	18	39080.00000000000	-1.0000000000000000
54	15	21	1644900.0000000000	-1.0000000000000000
55	15	19	11373.00000000000	-1.0000000000000000

transition data = 1 2 0
 check lenght of line list 55 85
 stop

PURE SCATTERING LIMIT (line and electron scattering) IN LTE WITH MORE LINES

Calculation of spectral energy distribution assuming that wind is consisting only of H.

Changing T_{eff} we will obtained different spectral distribution asuuming isothermal wind with $T_{\text{wind}}=5000\text{K}$.

